CompuShare

Un modelo de lenguaje abierto repartido entre varias computadoras. Informe del proyecto y de las mediciones al 4 de noviembre de 2026.

Todas las cifras salen de corridas reales y de sus registros. Lo que es cálculo se marca como tal. Sin servicios de OpenAI: todo es software libre (llama.cpp, Python, cloudflared).

93 de 93	**90 de 93**	**27,4 min**	**18,99 tok/s**
trozos de La metamorfosis copiados, 0 errores	copias idénticas letra por letra	el libro entero (119.123 caracteres)	generación, modelo de 9B en 2 computadoras

1. Qué es

CompuShare junta computadoras comunes para correr un solo modelo de lenguaje que ninguna corre bien sola. El modelo se parte por **bloques de capas**: cada máquina guarda y calcula su bloque, y por la red solo viaja el estado intermedio de cada palabra. El cálculo lo hace llama.cpp; CompuShare conecta las máquinas, elige cuáles usar, arranca el reparto con los ajustes que hacen falta por internet, se recupera de caídas y mide.

2. Cómo funciona

Pieza	Qué hace	Dónde
Relé	Punto de encuentro por WebSocket. Abre tubos (túneles TCP dentro del WebSocket) para que el maestro llegue a un trabajador sin abrir puertos en su router. Muestra el avance en /salud.	fabric/rele/rele.py
Trabajador	Arranca ggml-rpc-server solo en 127.0.0.1, lo ofrece por un tubo y lo relanza si se cae. Arma su caché de pesos desde su propio GGUF.	compushare unirse
Maestro	Espera trabajadores, mide la ida y vuelta de cada tubo, elige los menos y más cercanos, arranca llama-server --rpc y lo relanza si se cae.	compushare servir
Cliente	Habla con llama-server por su protocolo propio (/apply-template y /completion). transcribir espera si el servidor se cae, repite el trozo y puede retomar.	compushare probar, transcribir

3. La metamorfosis entera en Kaggle (4-nov-2026, después de los arreglos)

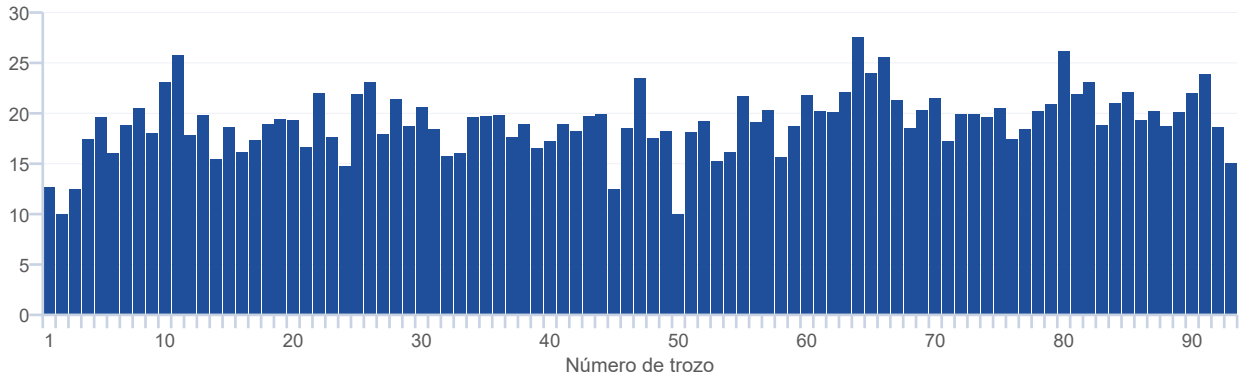
Qwen3.5-9B (Q8_0, 9,8 GB) repartido entre dos cuadernos de Kaggle, cada uno con 2 GPU Tesla T4, conectados por el relé y un túnel de Cloudflare. El modelo copia el libro trozo a trozo y cada copia se compara letra por letra con el original.

Medida	Antes de los arreglos	Después
Trozos terminados	42 de 93 (se cortó en el 43)	93 de 93, 0 errores
Copias exactas	39 de 42	90 de 93
Copias no exactas	trozos 1, 3, 35	trozos 1, 3, 35 (cambian comillas tipográficas por rectas)
Velocidad de generación	6,46 tok/s	18,99 tok/s
Lectura del prompt	47,4 tok/s	233,2 tok/s
Aciertos de la especulativa	88,8 %	89,7 %

Medida	Antes de los arreglos	Después
Segundos por trozo	57,1	17,7
Tiempo total	40,0 min para 42 trozos	27,4 min para el libro entero
Ida y vuelta maestro-trabajador	no se medía (ver sección 4)	37,1 ms con la conexión abierta
Abrir un tubo + saludo	664,1 ms (se informaba como ping)	667,1 ms
Pregunta corta de prueba (texto libre, primera petición)	1,1 tok/s, 382,9 KB por token	3,0 tok/s, 60,5 KB por token
Datos por el tubo al transcribir	14.910 MB (6,2 MB/s)	18.087 MB (11,0 MB/s)
Datos por el tubo al cargar el modelo	42,6 MB	42,2 MB

La meta era copiar el libro en menos de una hora: se copió entero en 27,4 minutos. La generación fue 2,9 veces más rápida que antes de los arreglos.

Tokens por segundo de generación en cada trozo (Kaggle, después de los arreglos)



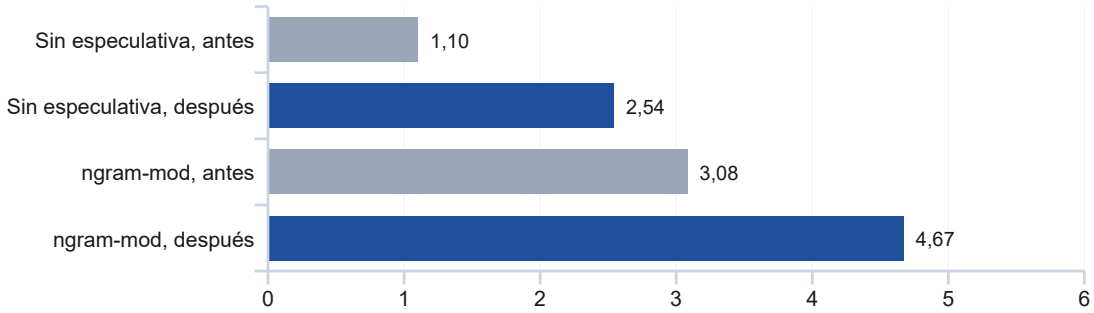
Mínimo 9,94, máximo 27,52 tok/s. Antes de los arreglos el rango era 3,61 a 7,69 tok/s.

4. Qué se arregló entre las dos corridas

Problema	Causa encontrada	Arreglo
Cada token rebotaba varias veces entre maestro y trabajador	El patrón que deja la tabla de salida en el maestro, (token_embd|output)\.weight, no tenía anclas y también atrapaba la proyección de salida de cada capa de atención (blk.N.attn_output.weight).	Patrón anclado: ^(token_embd|output)\.weight$. En local, por enlace y por token: de 707 KB y 4 idas y vueltas a 13,9 KB y 1; llama.cpp ahora reutiliza el grafo.
El ping informado era 664 ms	Se medía abriendo una conexión nueva: incluía abrir el tubo por el relé.	Se mide sobre la conexión abierta (37,1 ms en Kaggle) y aparte el tiempo de abrir el tubo (667,1 ms).
La corrida anterior se cortó en el trozo 43	No se cayó el trabajador (su ggml-rpc-server siguió vivo): el túnel cortó los WebSocket sin cuadro de cierre. El tubo se reconectó en 1 s, pero el maestro marcó como fallados los 50 trozos restantes.	Trabajador y maestro relanzan sus procesos; transcribir espera, repite el trozo y puede retomar. Probado en local matando el servidor de un trabajador: 6 de 6 trozos, listo de nuevo en 6 s.

5. Mediciones en una sola PC

tok/s con 5 trabajadores y 50 ms de ping simulado, mismo trozo y misma salida (Qwen3.5-0.8B)



Gris: antes de anclar el patrón. Azul: después. Medición local del 4-nov-2026, llama.cpp 11193, red simulada a 50 ms de ping y 50 Mb/s.

Qué	Resultado medido
Kaggle, 31-oct-2026: Qwen3.5-9B en 2 cuadernos, con el patrón sin anclas	6,8 a 6,9 tok/s; con MTP 10,3 a 10,8 tok/s (el ping de 237 ms que se informó era abrir el tubo)
Tabla de salida en el maestro	993 KB a 4,1 KB por palabra
Programa completo (servir + 2 unirse + transcribir) en una PC	3 trozos, 99,93 % de parecido
Caída de un trabajador a mitad de transcribir (local)	relanzado en 2 s, llama-server listo 4 s después; 6 de 6 trozos

6. Lo que todavía falta

- **Ancho de banda con la especulativa.** Durante la transcripción pasaron en promedio 11,0 MB/s (unos 88 Mb/s) por el tubo, unos 633 KB por token generado. En local se vio que, con especulativa, llama.cpp lee el estado recurrente entero de cada capa del trabajador al menos una vez por petición y vuelve a mandar el grafo cuando cambia el tamaño del lote. Para conexiones de casa hay que bajar esto.
- **La recuperación no se ejercitó en Kaggle:** en esta corrida no hubo cortes. Está probada en local.
- Probar con más de 2 computadoras y con computadoras caseras de verdad.
- Cambiar la clave de la red (FABRIC_CLAVE): quedó dentro de un cuaderno público.

7. Detalle por trozo (Kaggle, después de los arreglos)

Trozo	Caracteres	Tokens	tok/s	Aciertos	Segundos	Resultado
1	1.181	295	12,67	77,5 %	25,3	99,92 % parecido
2	415	102	9,94	63,7 %	11,2	exacto
3	1.476	363	12,43	80,9 %	31,0	99,93 % parecido
4	943	231	17,42	80,3 %	14,6	exacto
5	5.487	1300	19,61	97,7 %	70,2	exacto
6	1.017	239	16,02	83,2 %	16,2	exacto
7	1.397	325	18,81	92,6 %	18,8	exacto
8	1.416	332	20,51	94,7 %	17,7	exacto
9	1.295	295	18,00	83,7 %	17,4	exacto
10	1.092	274	23,10	96,0 %	13,2	exacto
11	619	154	25,71	100,0 %	7,0	exacto
12	1.169	290	17,82	82,2 %	17,8	exacto

Trozo	Caracteres	Tokens	tok/s	Aciertos	Segundos	Resultado
13	1.301	320	19,82	91,1 %	18,5	exacto
14	1.305	314	15,39	89,3 %	21,8	exacto
15	1.255	311	18,62	88,5 %	18,3	exacto
16	647	152	16,14	97,9 %	10,5	exacto
17	1.519	388	17,29	93,3 %	24,2	exacto
18	1.276	292	18,87	82,8 %	16,9	exacto
19	1.412	370	19,36	88,8 %	21,2	exacto
20	1.271	300	19,32	85,2 %	17,0	exacto
21	1.297	305	16,62	86,7 %	19,9	exacto
22	1.422	346	22,02	98,8 %	17,5	exacto
23	1.357	325	17,59	92,6 %	19,4	exacto
24	1.264	291	14,74	82,5 %	21,3	exacto
25	1.368	340	21,86	97,0 %	17,1	exacto
26	1.233	307	23,02	87,3 %	14,8	exacto
27	1.233	293	17,90	83,1 %	17,7	exacto
28	1.280	312	21,40	88,8 %	16,1	exacto
29	1.416	338	18,71	96,4 %	19,8	exacto
30	1.178	281	20,56	98,5 %	15,1	exacto
31	1.332	317	18,42	90,2 %	18,7	exacto
32	1.025	232	15,77	80,7 %	16,0	exacto
33	1.091	261	15,99	91,2 %	17,8	exacto
34	1.314	321	19,58	91,4 %	18,0	exacto
35	1.435	363	19,71	81,7 %	19,5	99,93 % parecido
36	1.177	292	19,76	82,8 %	15,6	exacto
37	1.058	252	17,57	88,0 %	15,7	exacto
38	1.191	265	18,88	92,7 %	15,5	exacto
39	1.496	367	16,51	88,1 %	24,1	exacto
40	1.333	297	17,26	84,3 %	19,0	exacto
41	1.392	315	18,87	89,6 %	18,2	exacto
42	1.498	352	18,22	84,3 %	20,9	exacto
43	1.469	337	19,68	96,2 %	18,7	exacto
44	1.013	246	19,87	85,8 %	13,6	exacto
45	983	226	12,46	78,5 %	19,5	exacto
46	1.253	288	18,51	81,7 %	17,1	exacto
47	1.347	311	23,45	88,5 %	14,2	exacto
48	542	134	17,48	85,6 %	8,7	exacto
49	1.311	311	18,23	88,5 %	18,0	exacto
50	420	92	9,96	56,8 %	10,4	exacto
51	1.103	252	18,09	86,2 %	14,8	exacto

Trozo	Caracteres	Tokens	tok/s	Aciertos	Segundos	Resultado
52	1.394	316	19,21	89,9 %	17,9	exacto
53	916	218	15,25	99,0 %	15,7	exacto
54	1.262	303	16,14	86,1 %	20,4	exacto
55	1.419	348	21,69	99,4 %	17,8	exacto
56	1.374	335	19,06	95,6 %	19,1	exacto
57	1.289	316	20,31	89,9 %	16,6	exacto
58	719	167	15,64	74,8 %	11,9	exacto
59	1.448	344	18,74	98,2 %	20,1	exacto
60	1.451	345	21,77	98,5 %	17,4	exacto
61	1.346	320	20,21	90,0 %	17,1	exacto
62	1.331	334	20,07	95,3 %	18,1	exacto
63	1.404	331	22,09	94,4 %	16,6	exacto
64	1.215	285	27,52	100,0 %	12,4	exacto
65	1.350	328	23,97	93,5 %	15,2	exacto
66	1.457	342	25,58	97,6 %	14,8	exacto
67	1.324	318	21,25	90,5 %	16,4	exacto
68	1.198	276	18,52	96,7 %	15,9	exacto
69	1.543	365	20,28	87,6 %	19,7	exacto
70	1.318	321	21,47	91,4 %	15,9	exacto
71	659	164	17,19	73,3 %	10,7	exacto
72	1.246	289	19,86	82,0 %	15,9	exacto
73	1.383	324	19,92	92,3 %	18,0	exacto
74	1.353	325	19,61	92,6 %	17,6	exacto
75	1.411	352	20,47	84,3 %	18,3	exacto
76	1.475	330	17,39	94,1 %	20,7	exacto
77	976	237	18,45	82,5 %	14,1	exacto
78	1.160	282	20,18	98,9 %	15,4	exacto
79	1.466	356	20,91	85,3 %	18,7	exacto
80	1.192	283	26,10	99,3 %	12,2	exacto
81	1.423	350	21,83	100,0 %	17,5	exacto
82	1.161	285	23,05	100,0 %	13,5	exacto
83	827	194	18,80	87,6 %	11,6	exacto
84	1.135	272	20,98	95,3 %	14,3	exacto
85	1.408	340	22,05	97,0 %	16,4	exacto
86	1.209	310	19,34	88,2 %	17,5	exacto
87	1.418	355	20,16	85,1 %	19,2	exacto
88	1.457	362	18,68	86,8 %	21,0	exacto
89	1.319	327	20,13	93,2 %	17,8	exacto
90	1.226	297	21,97	84,3 %	14,4	exacto

Trozo	Caracteres	Tokens	tok/s	Aciertos	Segundos	Resultado
91	1.345	333	23,85	95,0 %	15,5	exacto
92	1.462	373	18,57	89,6 %	21,9	exacto
93	1.330	305	14,99	86,7 %	21,4	exacto

Código: github.com/EnmanuelReynoso23/CompuShare (licencia MIT). Datos: fabric/kaggle/resultados/20261104-metamorfosis y 20261104-metamorfosis-antes (métricas sin el texto del libro).